

第13章: 通知システム

前の章（第12章: メール送信）では、メール送信の仕組みを学びました。

この章では **通知システム** を学びます。いいね、コメント、フォローなどのアクションをユーザーに知らせる仕組みです。

13.0 実習手順の進め方と手順マップ

手順に沿って進めると、**どのファイルに何を入力し、何を確認すればよいか** が分かります。形式の説明は [チュートリアルの進め方](#) を参照してください。

手順	主な対象ファイル（例）	完了時に確認すること
通知タイプ・モデル	<code>prisma/schema.prisma</code>	通知が DB に保存できる
通知作成・取得	<code>lib/actions/notification*.ts</code>	通知一覧・既読が動く
通知設定	<code>lib/actions/notification-preferences.ts</code>	種類別 ON/OFF が保存される
NotificationItem・List・Badge	<code>components/notification/*</code>	一覧・未読バッジが表示される
通知ページ	<code>app/(main)/notifications/page.tsx</code>	ページで通知が表示される

各セクションで **対象ファイル・入力するコード（サンプルコード）・実行方法・実行するとこうなる・このあとと変わること・確認方法** を確認しながら進めてください。

この章で学ぶこと

トピック	内容
通知タイプ	13種類の通知タイプとその用途
通知モデル	Prismaスキーマでの通知データ設計
通知の作成と削除	内部APIとしての通知CRUD
通知設定	ユーザーごとの通知プリファレンス
Server Actions	通知取得・既読マーク・未読数カウント
NotificationItem	個別通知の表示コンポーネント
NotificationList	無限スクロール付き通知一覧
NotificationBadge	未読数バッジとポーリング
通知ページ	SSR + クライアントハイドレーション

13.1 通知タイプ

BON-LOGでは、Prismaスキーマで **13種類** の通知タイプをenumで定義しています。

サンプルコード（実際のプロジェクトから）

```
// ファイル: prisma/schema.prisma

enum NotificationType {
  like           // 投稿へのいいね
  comment        // 投稿へのコメント
  follow         // フォロー
  quote          // 引用投稿
  reply          // コメントへの返信
  comment_like   // コメントへのいいね
  follow_request // フォローリクエスト（非公開アカウント）
  follow_request_approved // フォローリクエスト承認
  system         // システム通知
  mention        // メンション
  message        // メッセージ
  repost         // リポスト
  subscription_expiring // サブスクリプション期限切れ間近
}
```

ファイルパス: `prisma/schema.prisma`

各通知タイプの詳細

通知タイプ	トリガー	送信先	関連データ
<code>like</code>	投稿にいいね	投稿者	postId
<code>comment</code>	投稿にコメント	投稿者	postId, commentId
<code>follow</code>	ユーザーをフォロー	フォローされた人	なし
<code>quote</code>	投稿を引用	元投稿者	postId
<code>reply</code>	コメントに返信	元コメント者	postId, commentId
<code>comment_like</code>	コメントにいいね	コメント者	commentId
<code>follow_request</code>	非公開アカウントへフォロー申請	アカウント所有者	なし
<code>follow_request_approved</code>	フォロー申請を承認	申請者	なし
<code>system</code>	システムからの通知	対象ユーザー	なし
<code>mention</code>	メンションされた	メンション先	postId
<code>message</code>	メッセージ受信	受信者	なし
<code>repost</code>	リポストされた	元投稿者	postId
<code>subscription_expiring</code>	サブスク期限間近	対象ユーザー	なし

Server Actions側の型定義

Server Actions（`lib/actions/notification.ts`）では、実際にアプリ内で使用する **8種類** の通知タイプを型として定義しています。

```
// ファイル: lib/actions/notification.ts
```

```
export type NotificationType =  
  | 'like'  
  | 'comment'  
  | 'follow'  
  | 'quote'  
  | 'reply'  
  | 'comment_like'  
  | 'follow_request'  
  | 'follow_request_approved'
```

ファイルパス: `lib/actions/notification.ts`

これで何ができるようになるか

- Prismaの型安全な通知タイプ管理により、不正な通知タイプの保存を防止できる
- Server Actions側で使用する8タイプを絞り込み、将来の `system`、`mention` 等の拡張に備えている

これを省略するとどうなるか

- 通知タイプを文字列で管理すると、タイポにより不正な通知が生まれる
- データベースの整合性が保てなくなる

13.2 通知モデル設計

通知データのデータベース設計を見ていきます。

サンプルコード（実際のプロジェクトから）

```
// ファイル: prisma/schema.prisma

model Notification {
  id          String          @id @default(cuid())
  userId      String          @map("user_id")
  actorId     String          @map("actor_id")
  type        NotificationType
  postId      String?         @map("post_id")
  commentId   String?         @map("comment_id")
  isRead      Boolean         @default(false) @map("is_read")
  createdAt   DateTime        @default(now()) @map("created_at")

  // リレーション
  user      User      @relation("NotificationUser", fields: [userId], references: [id], onDelete: Cascade)
  actor     User      @relation("NotificationActor", fields: [actorId], references: [id], onDelete: Cascade)
  post      Post?     @relation(fields: [postId], references: [id], onDelete: Cascade)
  comment   Comment?  @relation(fields: [commentId], references: [id], onDelete: Cascade)

  // インデックス
  @@index([userId])
  @@index([userId, isRead])
  @@map("notifications")
}
```

ファイルパス: `prisma/schema.prisma`

期待される出力（テーブル構造）

notifications テーブル:

カラム	型	説明
id	String (cuid)	主キー
user_id	String	通知を受け取るユーザー
actor_id	String	アクションを行ったユーザー
type	NotificationType	通知の種類
post_id	String?	関連する投稿（任意）
comment_id	String?	関連するコメント（任意）
is_read	Boolean	既読フラグ（デフォルト: false）
created_at	DateTime	作成日時

設計のポイント

通知データの構造:

```
Notification
|
|  userId → 通知を受け取る人
|  actorId → アクションを起こした人
|  type → 何が起きたか
|  postId? → どの投稿に対して (オプション)
|  commentId? → どのコメントに対して (オプション)
|  isRead → 読んだかどうか
|
```

リレーション設計:

- `user`: 通知を受け取るユーザー (`NotificationUser` リレーション名)
- `actor`: 通知を発生させたユーザー (`NotificationActor` リレーション名)
- 同じ `User` モデルに対して2つのリレーションがあるため、リレーション名で区別

インデックス設計:

- `@@index([userId])`: ユーザーの通知一覧取得を高速化
- `@@index([userId, isRead])`: 未読通知のカウントを高速化

カスケード削除:

- `onDelete: Cascade` により、ユーザー・投稿・コメント削除時に関連通知も自動削除

これで何ができるようになるか

- 「誰が」「何を」「誰の投稿/コメントに対して」行ったかを一元管理できる
- インデックスにより、大量の通知があっても高速に取得できる

これを省略するとどうなるか

- インデックスがないと、通知が増えるほどページ表示が遅くなる
- カスケード削除がないと、削除済みユーザーの通知がゴミデータとして残り続ける

13.3 通知の作成（内部API）

通知の作成は、他のServer Actionsから呼び出される内部APIとして実装されています。

サンプルコード（実際のプロジェクトから）

```
// ファイル: lib/actions/notification.ts

'use server'

import { prisma } from '@lib/db'
import { auth } from '@lib/auth'
import { revalidatePath } from 'next/cache'
import { getMutedUserIds } from './filter-helper'
import { DEFAULT_PAGE_LIMIT } from '@lib/constants/limits'
import { ERR_AUTH_REQUIRED } from '@lib/constants/errors'

export type NotificationType =
  | 'like'
  | 'comment'
  | 'follow'
  | 'quote'
  | 'reply'
  | 'comment_like'
  | 'follow_request'
  | 'follow_request_approved'

export async function createNotification({
  userId,
  actorId,
  type,
  postId,
  commentId,
}: {
  userId: string
  actorId: string
  type: NotificationType
  postId?: string
  commentId?: string
}) {
  // ガード1: 自分自身への通知は作成しない
  if (userId === actorId) {
    return { success: true }
  }

  // ガード2: ブロック関係チェック（双方向）
  const block = await prisma.block.findFirst({
    where: {
      OR: [
        { blockerId: userId, blockedId: actorId },
        { blockerId: actorId, blockedId: userId },
      ],
    },
  },
})
}
```

```

if (block) {
  return { success: true }
}

// ガード3: 通知設定チェック (ユーザーがこのタイプの通知をオフにしているか)
const user = await prisma.user.findUnique({
  where: { id: userId },
  select: { notificationPreferences: true },
})
const prefs = (user?.notificationPreferences as Record<string, boolean>) || {}
if (prefs[type] === false) {
  return { success: true }
}

// ガード4: 重複チェック (同一内容の通知が既に存在するか)
const existingNotification = await prisma.notification.findFirst({
  where: {
    userId,
    actorId,
    type,
    postId: postId || null,
    commentId: commentId || null,
  },
})

if (existingNotification) {
  return { success: true }
}

// 通知を作成
await prisma.notification.create({
  data: {
    userId,
    actorId,
    type,
    postId,
    commentId,
  },
})

return { success: true }
}

```

ファイルパス: `lib/actions/notification.ts`

期待される出力（処理フロー）

createNotification() の処理フロー：

いいねアクション



自分自身へのアクション？ → Yes → 何もしない ✓

↓ No



ブロック関係がある？ → Yes → 何もしない ✓

↓ No



通知設定でOFFにしている？ → Yes → 何もしない ✓

↓ No



同じ通知が既に存在する？ → Yes → 何もしない ✓

↓ No



通知を作成 ✓

他のServer Actionsからの呼び出し例

```
// ファイル: lib/actions/comment.ts (コメント作成時の通知例)

// 動的インポートで通知モジュールを読み込み
const { createNotification } = await import('@lib/actions/notification')

// 返信通知を作成
await createNotification({
  userId: parentComment.userId, // 元コメントの投稿者
  actorId: session.user.id,      // 返信した人
  type: 'reply',
  postId: postId,
  commentId: newComment.id,
})
```

ファイルパス: `lib/actions/comment.ts`

これで何ができるようになるか

- 4段階のガード処理により、不要な通知の作成を防止できる
- 他のServer Actionsから簡潔に通知を作成できる
- 重複通知が防止されるため、ユーザー体験が損なわれない

これを省略するとどうなるか

- 自分自身の行動で通知が届いてしまう
- ブロックしている相手からの通知が届いてしまう
- 同じ通知が大量に生成されてしまう

13.4 通知の削除（内部API）

アクションの取り消し時（いいね解除、フォロー解除など）に、対応する通知を削除します。

サンプルコード（実際のプロジェクトから）

```
// ファイル: lib/actions/notification.ts

export async function deleteNotification({
  userId,
  actorId,
  type,
  postId,
  commentId,
}: {
  userId: string
  actorId: string
  type: NotificationType
  postId?: string
  commentId?: string
}) {
  await prisma.notification.deleteMany({
    where: {
      userId,
      actorId,
      type,
      postId: postId || null,
      commentId: commentId || null,
    },
  })
}

return { success: true }
}
```

ファイルパス: `lib/actions/notification.ts`

期待される出力

`deleteNotification()` の使用例:

いいね解除時:

```
deleteNotification({
  userId: "投稿者ID",
  actorId: "いいねした人ID",
  type: "like",
  postId: "投稿ID"
})
```

→ 該当する通知が削除される

→ 該当がなくてもエラーにならない (`deleteMany` の特性)

`deleteMany` を使う理由

- `delete`: 1件のみ削除。該当なしでエラー
- `deleteMany`: 条件に合う全件を削除。該当なしでも `{ count: 0 }` を返すだけでエラーにならない

これで何ができるようになるか

- いいね解除時に「いいね通知」も一緒に消せる
- フォロー解除時にフォロー通知も消せる
- 安全に呼び出せる (該当なしでもエラーにならない)

これを省略するとどうなるか

- いいねを解除しても通知が残り続け、ユーザーが混乱する

13.5 通知設定 (プリファレンス)

ユーザーごとに通知の受信設定を管理できます。

サンプルコード（実際のプロジェクトから）

```
// ファイル: lib/actions/notification-preferences.ts

'use server'

import { prisma } from '@lib/db'
import { revalidatePath } from 'next/cache'
import { requireAuth } from '@lib/actions/utils'

/**
 * 通知設定の型定義
 */
export type NotificationPreferences = {
  like?: boolean
  comment?: boolean
  reply?: boolean
  comment_like?: boolean
  follow?: boolean
  quote?: boolean
  follow_request?: boolean
  follow_request_approved?: boolean
}

/**
 * 通知設定を取得する
 */
export async function getNotificationPreferences(): Promise<{
  preferences?: NotificationPreferences
  error?: string
}> {
  const { userId, error: authError } = await requireAuth()
  if (!userId) return { error: authError! }

  const user = await prisma.user.findUnique({
    where: { id: userId },
    select: { notificationPreferences: true },
  })

  const preferences = (user?.notificationPreferences as NotificationPreferences) || {}
  return { preferences }
}

/**
 * 通知設定を更新する
 */
export async function updateNotificationPreferences(
  preferences: NotificationPreferences
): Promise<{ success?: boolean; error?: string }> {
  const { userId, error: authError } = await requireAuth()
  if (!userId) return { error: authError! }
```

```
// バリデーション: 許可されたキーのみ
const allowedKeys = [
  'like', 'comment', 'reply', 'comment_like',
  'follow', 'quote', 'follow_request', 'follow_request_approved'
]
const sanitized: Record<string, boolean> = {}
for (const [key, value] of Object.entries(preferences)) {
  if (allowedKeys.includes(key) && typeof value === 'boolean') {
    sanitized[key] = value
  }
}

await prisma.user.update({
  where: { id: userId },
  data: { notificationPreferences: sanitized },
})

revalidatePath('/settings/notifications')
return { success: true }
}
```

ファイルパス: `lib/actions/notification-preferences.ts`

期待される出力

通知設定の流れ:

ユーザーが設定画面でトグルを切り替え

↓

`updateNotificationPreferences({ like: false, comment: true })`

↓

バリデーション (許可されたキーかつboolean値のみ通過)

↓

`User.notificationPreferences` に JSON として保存

↓

`revalidatePath('/settings/notifications')` でキャッシュ更新

↓

次回 `createNotification` 時に:

`prefs['like'] === false` → 通知を作成しない ✓

通知設定と通知作成の連携

`createNotification` 内で以下のようにチェックされます。

```
// lib/actions/notification.ts の createNotification 内

const user = await prisma.user.findUnique({
  where: { id: userId },
  select: { notificationPreferences: true },
})
const prefs = (user?.notificationPreferences as Record<string, boolean>) || {}
if (prefs[type] === false) {
  return { success: true } // 通知を作成しない
}
```

これで何ができるようになるか

- ユーザーが不要な通知を個別にオフにできる
- いいね通知はOFF、コメント通知はON、といった柔軟な設定が可能
- `allowedKeys` によるバリデーションで不正なキーの混入を防止

これを省略するとどうなるか

- 全ての通知が強制的に届き、ユーザーが通知疲れを起こす
- 不正なキーでDBが汚染されるリスクがある

13.6 通知一覧の取得（Server Action）

通知一覧の取得は、カーソルベースのページネーションで実装されています。

サンプルコード（実際のプロジェクトから）

```
// ファイル: lib/actions/notification.ts

export async function getNotifications(cursor?: string, limit = DEFAULT_PAGE_LIMIT) {
  // 認証チェック
  const session = await auth()
  if (!session?.user?.id) {
    return { error: ERR_AUTH_REQUIRED, notifications: [], nextCursor: undefined }
  }

  // ミュートしているユーザーのIDリストを取得
  const mutedUserIds = await getMutedUserIds(session.user.id)

  // 通知一覧を取得
  const notifications = await prisma.notification.findMany({
    where: {
      userId: session.user.id,
      // ミュートしているユーザーからの通知を除外
      ... (mutedUserIds.length > 0 && {
        actorId: {
          notIn: mutedUserIds,
        },
      }),
    },
    include: {
      actor: {
        select: { id: true, nickname: true, avatarUrl: true },
      },
      post: {
        select: { id: true, content: true },
      },
      comment: {
        select: { id: true, content: true },
      },
    },
    orderBy: { createdAt: 'desc' },
    take: limit,
    ... (cursor && {
      cursor: { id: cursor },
      skip: 1,
    }),
  })

  return {
    notifications,
    nextCursor: notifications.length === limit
      ? notifications[notifications.length - 1]?.id
      : undefined,
  }
}
```

```
}  
}
```

ファイルパス: `lib/actions/notification.ts`

使用している定数

```
// ファイル: lib/constants/limits.ts  
  
/** デフォルトのページ取得件数 */  
export const DEFAULT_PAGE_LIMIT = 20
```

ファイルパス: `lib/constants/limits.ts`

期待される出力

getNotifications() のレスポンス例:

```
{
  notifications: [
    {
      id: "clx123 ... ",
      type: "like",
      isRead: false,
      createdAt: "2026-02-22T10:00:00Z",
      actor: {
        id: "user456",
        nickname: "盆栽太郎",
        avatarUrl: "https://example.com/avatar.jpg"
      },
      post: {
        id: "post789",
        content: "今日の黒松の手入れ ... "
      },
      comment: null
    },
    ... (最大20件)
  ],
  nextCursor: "clx999 ... " // 次のページがある場合
}
```

カーソルベースページネーションの仕組み

1ページ目: getNotifications()

- take: 20 で最新20件を取得
- nextCursor: 20件目のID

2ページ目: getNotifications("clx999 ... ")

- cursor: { id: "clx999 ... " }
- skip: 1 (カーソル自体をスキップ)
- take: 20 で次の20件を取得

最後のページ: notifications.length < 20

- nextCursor: undefined (次のページなし)

ミュートユーザーの除外

ミュートフィルタリングの流れ:

1. `getMutedUserIds(session.user.id)`
→ `["mutedUser1", "mutedUser2"]`
2. `mutedUserIds.length > 0` の場合のみ条件を追加
→ `actorId: { notIn: ["mutedUser1", "mutedUser2"] }`
3. ミュートユーザーからの通知は結果に含まれない

これで何ができるようになるか

- 効率的なカーソルベースページネーションで大量の通知を段階的に取得できる
- ミュートユーザーからの通知を自動除外できる
- `select` で必要最小限のフィールドのみ取得し、パフォーマンスを最適化

これを省略するとどうなるか

- ページネーションがないと、全通知を一度に取得してメモリを圧迫する
- ミュートフィルタリングがないと、ブロック・ミュート機能が形骸化する

13.7 既読マーク（個別・一括）

個別の既読マーク

```
// ファイル: lib/actions/notification.ts

export async function markAsRead(notificationId: string) {
  const session = await auth()
  if (!session?.user?.id) {
    return { error: ERR_AUTH_REQUIRED }
  }

  await prisma.notification.update({
    where: {
      id: notificationId,
      userId: session.user.id, // セキュリティ: 自分の通知のみ
    },
    data: { isRead: true },
  })

  revalidatePath('/notifications')

  return { success: true }
}
```

ファイルパス: `lib/actions/notification.ts`

一括の既読マーク

```
// ファイル: lib/actions/notification.ts

export async function markAllAsRead() {
  const session = await auth()
  if (!session?.user?.id) {
    return { error: ERR_AUTH_REQUIRED }
  }

  await prisma.notification.updateMany({
    where: {
      userId: session.user.id,
      isRead: false,
    },
    data: { isRead: true },
  })

  revalidatePath('/notifications')

  return { success: true }
}
```

ファイルパス: `lib/actions/notification.ts`

期待される出力

```
markAsRead("notif123"):
  → notification.isRead: false → true
  → revalidatePath('/notifications')
  → { success: true }

markAllAsRead():
  → 全未読通知を一括更新
  → updateMany は { count: 5 } のような結果を返す
  → revalidatePath('/notifications')
  → { success: true }
```

セキュリティのポイント

```
// where句にuserIdを含めることで、他人の通知を操作できないようにする
await prisma.notification.update({
  where: {
    id: notificationId,
    userId: session.user.id, // ← これが重要
  },
  data: { isRead: true },
})
```

- `userId: session.user.id` を where 条件に含めることで、認証済みユーザー自身の通知のみ操作可能
- 他人の通知IDを指定しても更新されない

これで何ができるようになるか

- 通知クリック時に個別に既読にできる
- 通知ページ表示時に一括既読にできる
- セキュリティ面で他人の通知を操作するリクエストを遮断できる

これを省略するとどうなるか

- 未読バッジが永遠に消えない
- 他人の通知を既読にできるセキュリティホールが生まれる

13.8 未読数の取得

ヘッダーの通知バッジに表示する未読通知数を取得します。

サンプルコード（実際のプロジェクトから）

```
// ファイル: lib/actions/notification.ts

export async function getUnreadCount() {
  const session = await auth()

  if (!session?.user?.id) {
    return { count: 0 }
  }

  // ミュートしているユーザーからの通知は未読数にカウントしない
  const mutedUserIds = await getMutedUserIds(session.user.id)

  const count = await prisma.notification.count({
    where: {
      userId: session.user.id,
      isRead: false,
      ...(mutedUserIds.length > 0 && {
        actorId: {
          notIn: mutedUserIds,
        },
      }),
    },
  })

  return { count }
}
```

ファイルパス: `lib/actions/notification.ts`

期待される出力

```
getUnreadCount():
  未ログイン → { count: 0 }
  ログイン済み（未読3件） → { count: 3 }
  ログイン済み（未読0件） → { count: 0 }
  ログイン済み（ミュート除外後） → { count: 2 }
```

count() vs findMany().length

```
prisma.notification.count()    → SQLで COUNT(*) を実行 (高速)
prisma.notification.findMany() → 全レコードを取得してから .length (非効率)
```

`count()` はデータベース側でカウントするため、通知が何千件あっても高速に結果を返します。

これで何ができるようになるか

- ヘッダーの通知バッジに正確な未読数を表示できる
- ミュートユーザーの通知をカウントから除外できる
- 未ログイン時にエラーにならず安全に `0` を返せる

これを省略するとどうなるか

- 未読数がわからず、通知の存在に気づけない
- ミュートしたユーザーの通知もカウントされてしまう

13.9 NotificationItem コンポーネント

個別の通知を表示するクライアントコンポーネントです。

サンプルコード（実際のプロジェクトから）

```
// ファイル: components/notification/NotificationItem.tsx

'use client'

import Link from 'next/link'
import Image from 'next/image'
import { formatDistanceToNow } from 'date-fns'
import { ja } from 'date-fns/locale'
import { markAsRead } from '@lib/actions/notification'

type Notification = {
  id: string
  type: string
  isRead: boolean
  createdAt: Date | string
  actor: {
    id: string
    nickname: string
    avatarUrl: string | null
  }
  post?: {
    id: string
    content: string | null
  } | null
  comment?: {
    id: string
    content: string | null
  } | null
}

type NotificationItemProps = {
  notification: Notification
}
```

ファイルパス: `components/notification/NotificationItem.tsx`

アイコンの種類別表示

通知タイプごとに異なるカスタムSVGアイコンを表示します。

```
// ファイル: components/notification/NotificationItem.tsx

// ハートアイコン (いいね通知用)
function HeartIcon({ className }: { className?: string }) {
  return (
    <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="currentColor" className={className}>
      <path d="M19 14c1.49-1.46 3-3.21 3-5.5A5.5 5.5 0 0 0 16.5 3c-1.76 0-3 .5-4.5 2-1.5-1.5-3-3.21-3-5.5A5.5 5.5 0 0 0 13.5 3c1.76 0 3 .5 4.5 2" />
    </svg>
  )
}

// 通知タイプ → アイコン + 色 の対応表
function getNotificationIcon(type: string) {
  switch (type) {
    case 'like':
    case 'comment_like':
      return <HeartIcon className="w-5 h-5 text-red-500" />
    case 'comment':
      return <MessageCircleIcon className="w-5 h-5 text-blue-500" />
    case 'follow':
    case 'follow_request_approved':
      return <UserPlusIcon className="w-5 h-5 text-green-500" />
    case 'follow_request':
      return <UserPlusIcon className="w-5 h-5 text-yellow-500" />
    case 'quote':
    case 'repost':
      return <RepeatIcon className="w-5 h-5 text-purple-500" />
    case 'reply':
      return <ReplyIcon className="w-5 h-5 text-orange-500" />
    default:
      return <MessageCircleIcon className="w-5 h-5 text-muted-foreground" />
  }
}
```

ファイルパス: `components/notification/NotificationItem.tsx`

アイコンカラー一覧

通知タイプ	アイコン	色
like , comment_like	HeartIcon	赤 (text-red-500)
comment	MessageCircleIcon	青 (text-blue-500)
follow , follow_request_approved	UserPlusIcon	緑 (text-green-500)
follow_request	UserPlusIcon	黄 (text-yellow-500)
quote , repost	RepeatIcon	紫 (text-purple-500)
reply	ReplyIcon	オレンジ (text-orange-500)
その他	MessageCircleIcon	グレー (text-muted-foreground)

通知メッセージの生成

```
// ファイル: components/notification/NotificationItem.tsx

function getNotificationMessage(type: string, actorName: string) {
  switch (type) {
    case 'like':
      return <<strong>{actorName}</strong>さんがあなたの投稿にいいねしました</>
    case 'comment_like':
      return <<strong>{actorName}</strong>さんがあなたのコメントにいいねしました</>
    case 'comment':
      return <<strong>{actorName}</strong>さんがあなたの投稿にコメントしました</>
    case 'follow':
      return <<strong>{actorName}</strong>さんがあなたをフォローしました</>
    case 'follow_request':
      return <<strong>{actorName}</strong>さんからフォローリクエストが届きました</>
    case 'follow_request_approved':
      return <<strong>{actorName}</strong>さんがフォローリクエストを承認しました</>
    case 'quote':
      return <<strong>{actorName}</strong>さんがあなたの投稿を引用しました</>
    case 'repost':
      return <<strong>{actorName}</strong>さんがあなたの投稿をリポストしました</>
    case 'reply':
      return <<strong>{actorName}</strong>さんがあなたのコメントに返信しました</>
    default:
      return <<strong>{actorName}</strong>さんからの通知</>
  }
}
```

ファイルパス: `components/notification/NotificationItem.tsx`

リンク先の決定

```
// ファイル: components/notification/NotificationItem.tsx

function getNotificationLink(notification: Notification) {
  const { type, post, comment, actor } = notification

  // フォロー関連通知 → ユーザーページ
  if (type === 'follow' || type === 'follow_request_approved') {
    return `/users/${actor.id}`
  }

  // フォローリクエスト → フォローリクエスト管理ページ
  if (type === 'follow_request') {
    return '/settings/follow-requests'
  }

  // 投稿がある場合
  if (post) {
    // コメントがある場合はアンカーリンク
    if (comment) {
      return `/posts/${post.id}#comment-${comment.id}`
    }
    return `/posts/${post.id}`
  }

  // デフォルトはユーザーページ
  return `/users/${actor.id}`
}
```

ファイルパス: `components/notification/NotificationItem.tsx`

メインコンポーネント

```
// ファイル: components/notification/NotificationItem.tsx

export function NotificationItem({ notification }: NotificationItemProps) {
  const timeAgo = formatDistanceToNow(new Date(notification.createdAt), {
    addSuffix: true,
    locale: ja,
  })

  const handleClick = async () => {
    if (!notification.isRead) {
      await markAsRead(notification.id)
    }
  }

  const link = getNotificationLink(notification)
  const contentPreview = notification.comment?.content || notification.post?.content

  return (
    <Link
      href={link}
      onClick={handleClick}
      className={`flex gap-3 p-4 hover:bg-muted/50 transition-colors border-b ${
        !notification.isRead ? 'bg-primary/5' : ''
      }`}
    >
      {/* アバター + タイプアイコン */}
      <div className="flex-shrink-0 w-10 h-10 relative">
        {notification.actor.avatarUrl ? (
          <Image
            src={notification.actor.avatarUrl}
            alt={notification.actor.nickname}
            width={40}
            height={40}
            className="rounded-full object-cover"
          />
        ) : (
          <div className="w-10 h-10 rounded-full bg-muted flex items-center justify-center">
            <span className="text-muted-foreground text-sm">
              {notification.actor.nickname.charAt(0)}
            </span>
          </div>
        )}
      </div>
      {/* コンテンツ */}
      <div className="flex-1 min-w-0">
```

```

<p className="text-sm">
  {getNotificationMessage(notification.type, notification.actor.nickname)}
</p>
{contentPreview && (
  <p className="text-sm text-muted-foreground mt-1 line-clamp-1">
    {contentPreview}
  </p>
)}
<p className="text-xs text-muted-foreground mt-1">{timeAgo}</p>
</div>

{/* 未読インジケータ */}
{!notification.isRead && (
  <div className="flex-shrink-0">
    <div className="w-2 h-2 rounded-full bg-primary" />
  </div>
)}
</Link>
)
}

```

ファイルパス: `components/notification/NotificationItem.tsx`

期待される出力（表示例）

未読の通知:

[アバター]	盆栽太郎さんがあなたの投稿にいいねしました	•
♥	今日の黒松の手入れが ...	
	3分前	

背景色: `bg-primary/5`（薄い色で未読を示す）

既読の通知:

[アバター]	松田花子さんがあなたをフォローしました	
+		
	2時間前	

背景色なし、未読インジケータなし

これで何ができるようになるか

- 通知タイプに応じた色分けされたアイコンで一目でわかるUI
- クリック時に自動的に既読処理が行われる
- 相対時間表示（「3分前」「2時間前」）で直感的に理解できる
- 投稿内容のプレビュー表示で、どの投稿への通知かわかる

これを省略するとどうなるか

- 全ての通知が同じ見た目になり、種類を区別できない
- クリックしても既読にならず、何度もクリックしてしまう

13.10 NotificationList コンポーネント

無限スクロール付きの通知一覧コンポーネントです。React QueryのuseInfiniteQueryを使用します。

サンプルコード（実際のプロジェクトから）

```
// ファイル: components/notification/NotificationList.tsx

'use client'

import { useInfiniteQuery, useQueryClient } from '@tanstack/react-query'
import { useInView } from 'react-intersection-observer'
import { useEffect } from 'react'
import { NotificationItem } from './NotificationItem'
import { getNotifications, markAllAsRead } from '@lib/actions/notification'
import { DEFAULT_PAGE_LIMIT, SKELETON_COUNT } from '@lib/constants/limits'

// eslint-disable-next-line @typescript-eslint/no-explicit-any
type Notification = any

type NotificationListProps = {
  initialNotifications: Notification[]
}
```

ファイルパス: `components/notification/NotificationList.tsx`

無限スクロールの実装

```
// ファイル: components/notification/NotificationList.tsx

export function NotificationList({ initialNotifications }: NotificationListProps) {
  // Intersection Observer: 末尾要素がビューポートに入ったか監視
  const { ref, inView } = useInView()

  // React Query クライアント (キャッシュ操作)
  const queryClient = useQueryClient()

  // 無限スクロール用のReact Queryフック
  const {
    data,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage,
    isLoading,
    refetch,
  } = useInfiniteQuery({
    queryKey: ['notifications'],

    queryFn: async ({ pageParam }) => {
      return await getNotifications(pageParam)
    },

    initialPageParam: undefined as string | undefined,

    // SSRで取得したデータを初期値として設定
    initialData: {
      pages: [{
        notifications: initialNotifications,
        nextCursor: initialNotifications.length ≥ DEFAULT_PAGE_LIMIT
          ? initialNotifications[initialNotifications.length - 1]?.id
          : undefined,
      }],
      pageParams: [undefined],
    },

    getNextPageParam: (lastPage) => lastPage.nextCursor,
  })

  // 末尾がビューポートに入ったら次のページを取得
  useEffect(() => {
    if (inView && hasNextPage && !isFetchingNextPage) {
      fetchNextPage()
    }
  }, [inView, hasNextPage, isFetchingNextPage, fetchNextPage])

  // ページ表示時に自動で全通知を既読にする
  useEffect(() => {
```

```

const autoMarkAsRead = async () => {
  await markAllAsRead()
  refetch()
  queryClient.invalidateQueries({ queryKey: ['unreadCount'] })
}
autoMarkAsRead()
}, [refetch, queryClient])

// 「すべて既読にする」ボタンのハンドラ
const handleMarkAllAsRead = async () => {
  await markAllAsRead()
  refetch()
  queryClient.invalidateQueries({ queryKey: ['unreadCount'] })
}

if (isLoading) {
  return <NotificationListSkeleton />
}

const allNotifications = data?.pages.flatMap((page) => page.notifications) || []
const hasUnread = allNotifications.some((n) => !n.isRead)

// 通知がない場合
if (allNotifications.length === 0) {
  return (
    <div className="text-center py-12">
      <div className="w-16 h-16 mx-auto mb-4 rounded-full bg-muted flex items-center justify-center">
        <BellIcon className="w-8 h-8 text-muted-foreground" />
      </div>
      <h3 className="text-lg font-semibold mb-2">通知はありません</h3>
      <p className="text-muted-foreground">
        いいね、コメント、フォローなどの通知がここに表示されます
      </p>
    </div>
  )
}

return (
  <div>
    {/* すべて既読にするボタン */}
    {hasUnread && (
      <div className="flex justify-end p-2 border-b">
        <button
          onClick={handleMarkAllAsRead}
          className="flex items-center gap-1 text-sm text-primary hover:underline"
        >
          <CheckCheckIcon className="w-4 h-4" />
          すべて既読にする
        </button>
      </div>
    )}
  </div>
  {/* 通知一覧 */}

```

```

<div>
  {allNotifications.map((notification) => (
    <NotificationItem key={notification.id} notification={notification} />
  ))}
</div>

{/* 無限スクロール検知エリア */}
<div ref={ref} className="py-4 flex justify-center">
  {isFetchingNextPage && (
    <div className="flex items-center gap-2 text-muted-foreground">
      <div className="animate-spin h-4 w-4 border-2 border-primary border-t-transparent" />
      <span className="text-sm">読み込み中 ... </span>
    </div>
  )}
  {!hasNextPage && allNotifications.length > 0 && (
    <p className="text-sm text-muted-foreground">これ以上通知はありません</p>
  )}
</div>
</div>
)
}

```

ファイルパス: `components/notification/NotificationList.tsx`

スケルトンコンポーネント

```

// ファイル: components/notification/NotificationList.tsx

function NotificationListSkeleton() {
  return (
    <div>
      {[...Array(SKELETON_COUNT)].map((_, i) => (
        <div key={i} className="flex gap-3 p-4 border-b animate-pulse">
          <div className="w-10 h-10 rounded-full bg-muted" />
          <div className="flex-1 space-y-2">
            <div className="h-4 w-3/4 bg-muted rounded" />
            <div className="h-3 w-1/2 bg-muted rounded" />
          </div>
        </div>
      ))}
    </div>
  )
}

```

ファイルパス: `components/notification/NotificationList.tsx`

使用している定数

```
// ファイル: lib/constants/limits.ts

export const DEFAULT_PAGE_LIMIT = 20 // 1ページあたりの取得件数
export const SKELETON_COUNT = 5      // スケルトンの表示件数
```

ファイルパス: `lib/constants/limits.ts`

期待される出力（処理フロー）

NotificationList の動作フロー：

1. 初期表示

```
| SSRで取得した initialNotifications |  
| をそのまま表示（ちらつきなし） |
```

|



2. 自動既読処理（useEffect）

```
| markAllAsRead() → 全通知を既読に |  
| refetch() → UIを更新 |  
| invalidateQueries(['unreadCount']) |  
| → 通知バッジを0に更新 |
```

|



3. スクロールして末尾に到達

```
| inView: true |  
| hasNextPage: true |  
| → fetchNextPage() |  
| → getNotifications(nextCursor) |  
| → 追加の通知を表示 |
```

|



4. 全データ読み込み完了

```
| hasNextPage: false |  
| → 「これ以上通知はありません」 |
```

useInfiniteQuery のキーポイント

プロパティ	役割
<code>queryKey</code>	<code>['notifications']</code> でキャッシュキーを指定
<code>queryFn</code>	<code>getNotifications(pageParam)</code> でデータ取得
<code>initialData</code>	SSRデータを初期値に設定（ハイドレーション）
<code>initialPageParam</code>	最初のページはカーソルなし（ <code>undefined</code> ）
<code>getNextPageParam</code>	次のカーソルを返す関数
<code>fetchNextPage</code>	次のページを取得する関数
<code>hasNextPage</code>	次のページの有無

これで何ができるようになるか

- SSRデータを初期値にすることで、ページ表示時のちらつきを防止
- 無限スクロールで自然なUXを提供
- 通知ページを開くだけで全通知が既読になり、バッジもリセットされる
- スケルトンUIで読み込み中の体験を向上

これを省略するとどうなるか

- 全通知を一度に読み込み、パフォーマンスが低下する
- 既読処理を手動で行う必要があり、UXが悪化する

13.11 NotificationBadge コンポーネント

ナビゲーションバーに表示される未読通知数バッジです。React Queryのポーリング機能で定期更新します。

サンプルコード（実際のプロジェクトから）

```
// ファイル: components/notification/NotificationBadge.tsx

'use client'

import { useQuery } from '@tanstack/react-query'
import { getUnreadCount } from '@lib/actions/notification'
import { BADGE_OVERFLOW_THRESHOLD, REFETCH_INTERVAL_MS } from '@lib/constants/limits'

type NotificationBadgeProps = {
  className?: string
}

export function NotificationBadge({ className }: NotificationBadgeProps) {
  const { data } = useQuery({
    queryKey: ['unreadCount'],
    queryFn: async () => {
      return await getUnreadCount()
    },
    refetchInterval: REFETCH_INTERVAL_MS, // 30秒ごとに更新
  })

  const count = data?.count || 0

  if (count === 0) {
    return null
  }

  return (
    <span
      className={`inline-flex items-center justify-center min-w-[18px] h-[18px] px-1 text-
    >
      {count > BADGE_OVERFLOW_THRESHOLD ? `${BADGE_OVERFLOW_THRESHOLD}+` : count}
    </span>
  )
}
```

ファイルパス: `components/notification/NotificationBadge.tsx`

サーバーコンポーネント版

```
// ファイル: components/notification/NotificationBadge.tsx

export async function NotificationBadgeServer() {
  const { count } = await getUnreadCount()

  if (count === 0) {
    return null
  }

  return (
    <span className="inline-flex items-center justify-center min-w-[18px] h-[18px] px-1 text-center">
      {count > BADGE_OVERFLOW_THRESHOLD ? `${BADGE_OVERFLOW_THRESHOLD}+` : count}
    </span>
  )
}
```

ファイルパス: `components/notification/NotificationBadge.tsx`

使用している定数

```
// ファイル: lib/constants/limits.ts

export const REFETCH_INTERVAL_MS = 30000 // 30秒ごとにポーリング
export const BADGE_OVERFLOW_THRESHOLD = 99 // 99件を超えたら「99+」表示
```

ファイルパス: `lib/constants/limits.ts`

期待される出力

未読通知数に応じた表示:

未読0件: (何も表示しない)

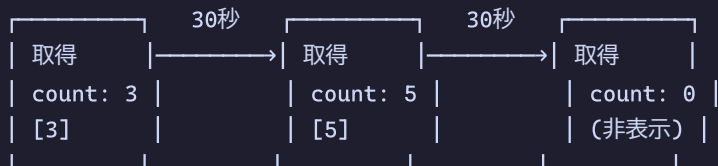
未読3件: [3] ← 赤い丸バッジに数字

未読15件: [15] ← 赤い丸バッジに数字

未読100件: [99+] ← BADGE_OVERFLOW_THRESHOLD を超えた場合

ポーリングの仕組み

React Query のポーリング:



`refetchInterval: 30000ms (REFETCH_INTERVAL_MS)`

※ 通知ページを開いた時は `invalidateQueries` で即座に更新

クライアント版 vs サーバー版の使い分け

	NotificationBadge	NotificationBadgeServer
種類	クライアントコンポーネント	サーバーコンポーネント
ポーリング	30秒ごとに自動更新	なし (初回レンダリング時のみ)
キャッシュ無効化	<code>invalidateQueries</code> に対応	非対応
用途	ナビゲーションバー (動的更新が必要)	SSR初期表示

これで何ができるようになるか

- 30秒ごとにバックグラウンドで未読数を確認し、リアルタイムに近い更新を提供
- 99件を超える場合は「99+」と表示してUI崩れを防止
- 通知ページで既読にした後、バッジが即座にリセットされる

これを省略するとどうなるか

- ユーザーが新着通知に気づけない
- 手動でページリロードしないと未読数が更新されない

13.12 通知ページ

通知ページはSSR（Server Component）とCSR（Client Component）を組み合わせて実装されています。

ページコンポーネント（Server Component）

```
// ファイル: app/(main)/notifications/page.tsx

import { getNotifications } from '@lib/actions/notification'
import { NotificationList } from '@components/notification/NotificationList'

export const metadata = {
  title: '通知 - BON-LOG',
}

export default async function NotificationsPage() {
  // サーバーサイドで通知一覧を取得
  const { notifications } = await getNotifications()

  return (
    <div className="bg-card rounded-lg border overflow-hidden">
      <div className="p-4 border-b">
        <h1 className="text-xl font-bold">通知</h1>
      </div>

      {/* 初期データを渡してクライアントコンポーネントで表示 */}
      <NotificationList initialNotifications={notifications || []} />
    </div>
  )
}
```

ファイルパス: `app/(main)/notifications/page.tsx`

ローディングスケルトン

```
// ファイル: app/(main)/notifications/loading.tsx

export default function NotificationsLoading() {
  return (
    <div className="bg-card rounded-lg border overflow-hidden">
      <div className="p-4 border-b">
        <div className="h-7 w-20 bg-muted rounded animate-pulse" />
      </div>

      <div>
        {[ ... Array(5) ].map( (_, i) => (
          <div key={i} className="flex gap-3 p-4 border-b animate-pulse">
            <div className="w-10 h-10 rounded-full bg-muted" />
            <div className="flex-1 space-y-2">
              <div className="h-4 w-3/4 bg-muted rounded" />
              <div className="h-3 w-1/2 bg-muted rounded" />
            </div>
          </div>
        ))}
      </div>
    </div>
  )
}
```

ファイルパス: `app/(main)/notifications/loading.tsx`

エラーバウンダリ

```
// ファイル: app/(main)/notifications/error.tsx

'use client'

import { PageError } from '@components/common/PageError'

export default function NotificationsError({
  error,
  reset,
}: {
  error: Error & { digest?: string }
  reset: () => void
}) {
  return (
    <PageError
      error={error}
      reset={reset}
      title="通知を読み込めません"
      description="通知の取得に失敗しました。再試行してください。"
      icon={
        <svg
          className="w-8 h-8 text-destructive"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}
              d="M15 17h5l-1.405-1.405A2.032 2.032 0 0118 14.158V11a6.002 6.002 0 00-4-5.659"
            />
          </svg>
        >
      )
    >
  )
}
```

ファイルパス: `app/(main)/notifications/error.tsx`

期待される出力（SSR + CSRの連携）

通知ページのレンダリングフロー：

1. ページ遷移開始

```
| loading.tsx が表示される |
| (スケルトンUI)          |
```



2. サーバーでデータ取得完了

```
| page.tsx (Server Component) |
| getNotifications() で初期データ取得 |
| → NotificationList に渡す   |
```



3. クライアントでハイドレーション

```
| NotificationList (Client Component) |
| initialData でSSRデータを使用      |
| → 即座に表示 (ちらつきなし)        |
| → markAllAsRead() で自動既読        |
| → ポーリング開始                    |
```

App Routerファイル規約

```
app/(main)/notifications/
├─ page.tsx      # メインページ (Server Component)
├─ loading.tsx   # Suspenseフォールバック (スケルトン)
└─ error.tsx     # エラーバウンダリ (Client Component、'use client' 必須)
```

これで何ができるようになるか

- SSRで初期データを取得し、SEOにも優しい通知ページを構築
- loading.tsxでページ遷移時のローディング体験を向上
- error.tsxでエラー発生時にもユーザーに適切なフィードバックと再試行ボタンを提供

これを省略するとどうなるか

- loading.tsxがないと、データ取得中に白い画面が表示される
- error.tsxがないと、エラー時にアプリ全体がクラッシュする可能性がある

13.13 Redis キャッシュ（Upstash Redis）

BON-LOGでは、キャッシュやレート制限にRedisを使用しています。本番環境ではUpstash Redis、開発環境ではインメモリフォールバックを使用します。

サンプルコード（実際のプロジェクトから）

```
// ファイル: lib/redis.ts

import { Redis } from '@upstash/redis'
import { IN_MEMORY_CLEANUP_THRESHOLD } from '@lib/constants/limits'
import logger from '@lib/logger'

/**
 * Redis互換ストアのインターフェース
 */
interface RedisLikeStore {
  get(key: string): Promise<string | null>
  set(key: string, value: string, options?: { ex?: number }): Promise<void>
  del(key: string): Promise<void>
  incr(key: string): Promise<number>
  expire(key: string, seconds: number): Promise<void>
  ttl(key: string): Promise<number>
}
```

ファイルパス: `lib/redis.ts`

インメモリストア（開発/テスト用）

```
// ファイル: lib/redis.ts

class InMemoryStore implements RedisLikeStore {
  private store = new Map<string, { value: string; expiresAt: number | null }>()

  private cleanExpired() {
    const now = Date.now()
    for (const [key, entry] of this.store.entries()) {
      if (entry.expiresAt && entry.expiresAt < now) {
        this.store.delete(key)
      }
    }
  }

  private static CLEANUP_THRESHOLD = IN_MEMORY_CLEANUP_THRESHOLD

  async get(key: string): Promise<string | null> {
    const entry = this.store.get(key)
    if (!entry) return null
    if (entry.expiresAt && entry.expiresAt < Date.now()) {
      this.store.delete(key)
      return null
    }
    return entry.value
  }

  async set(key: string, value: string, options?: { ex?: number }): Promise<void> {
    if (this.store.size > InMemoryStore.CLEANUP_THRESHOLD) {
      this.cleanExpired()
    }
    const expiresAt = options?.ex ? Date.now() + options.ex * 1000 : null
    this.store.set(key, { value, expiresAt })
  }

  async del(key: string): Promise<void> {
    this.store.delete(key)
  }

  async incr(key: string): Promise<number> {
    const entry = this.store.get(key)
    const currentValue = entry ? parseInt(entry.value, 10) || 0 : 0
    const newValue = currentValue + 1
    this.store.set(key, { value: newValue.toString(), expiresAt: entry?.expiresAt ?? null })
    return newValue
  }

  async expire(key: string, seconds: number): Promise<void> {
    const entry = this.store.get(key)
    if (entry) {

```

```
        entry.expiresAt = Date.now() + seconds * 1000
    }
}

async ttl(key: string): Promise<number> {
    const entry = this.store.get(key)
    if (!entry || !entry.expiresAt) return -1
    const remaining = Math.ceil((entry.expiresAt - Date.now()) / 1000)
    return remaining > 0 ? remaining : -2
}
}
```

ファイルパス: `lib/redis.ts`

Upstash Redisストア（本番用）

```
// ファイル: lib/redis.ts

class UpstashRedisStore implements RedisLikeStore {
  private client: Redis

  constructor(url: string, token: string) {
    this.client = new Redis({ url, token })
  }

  async get(key: string): Promise<string | null> {
    const result = await this.client.get<string>(key)
    return result
  }

  async set(key: string, value: string, options?: { ex?: number }): Promise<void> {
    if (options?.ex) {
      await this.client.set(key, value, { ex: options.ex })
    } else {
      await this.client.set(key, value)
    }
  }

  async del(key: string): Promise<void> {
    await this.client.del(key)
  }

  async incr(key: string): Promise<number> {
    return await this.client.incr(key)
  }

  async expire(key: string, seconds: number): Promise<void> {
    await this.client.expire(key, seconds)
  }

  async ttl(key: string): Promise<number> {
    return await this.client.ttl(key)
  }
}
```

ファイルパス: lib/redis.ts

シングルトンパターンとエクスポート

```
// ファイル: lib/redis.ts

let redisClient: RedisLikeStore | null = null

export function getRedisClient(): RedisLikeStore {
  if (redisClient) return redisClient

  const redisUrl = process.env.UPSTASH_REDIS_REST_URL
  const redisToken = process.env.UPSTASH_REDIS_REST_TOKEN

  if (redisUrl && redisToken) {
    logger.log('Using Upstash Redis')
    redisClient = new UpstashRedisStore(redisUrl, redisToken)
  } else {
    logger.log('Using in-memory store (Redis not configured)')
    redisClient = new InMemoryStore()
  }

  return redisClient
}

export const redis = {
  get client() {
    return getRedisClient()
  },
}
```

ファイルパス: `lib/redis.ts`

使用している定数

```
// ファイル: lib/constants/limits.ts

export const IN_MEMORY_CLEANUP_THRESHOLD = 1000 // ストアが1000エントリを超えたらクリーンアップ
```

ファイルパス: `lib/constants/limits.ts`

期待される出力

環境に応じた自動切り替え:

本番環境 (Vercel + Upstash) :

```
UPSTASH_REDIS_REST_URL="https://xxx.upstash.io"
UPSTASH_REDIS_REST_TOKEN="AYxx ..."
→ "Using Upstash Redis" がログ出力
→ HTTP REST API経由でRedisに接続
```

開発環境 (ローカル) :

```
環境変数なし
→ "Using in-memory store (Redis not configured)" がログ出力
→ JavaScriptのMapで代替
```

使用例:

```
import { redis } from '@lib/redis'

await redis.client.set('key', 'value', { ex: 60 }) // 60秒で期限切れ
const value = await redis.client.get('key')        // → 'value'
await redis.client.incr('counter')                  // → 1
await redis.client.del('key')                       // 削除
```

インターフェースによる抽象化の利点

RedisLikeStore インターフェース



- 呼び出し側はどちらの実装かを意識する必要がない
- テスト時はモックに置き換え可能
- 将来別のRedisクライアントに切り替えても呼び出し側は変更不要

これで何ができるようになるか

- セッション管理、レート制限、キャッシュなどの高速データストアとして活用
- 開発環境でRedisをセットアップしなくても動作する
- 本番環境ではUpstash Redisによる高信頼性のキャッシュを利用

これを省略するとどうなるか

- レート制限が実装できず、スパム攻撃に対して脆弱になる
- 開発環境でRedisの設定が必須になり、開発の敷居が上がる

13.14 メール通知

BON-LOGでは、メール通知機能も備えています。開発環境ではコンソール出力、本番環境ではResendを使用します。

サンプルコード（実際のプロジェクトから）

```
// ファイル: lib/email/index.ts

import logger from '@lib/logger'
import { Resend } from 'resend'

export interface EmailOptions {
  to: string
  subject: string
  html: string
  text?: string
}

export interface EmailResult {
  success: boolean
  messageId?: string
  error?: string
}

interface EmailProvider {
  send(options: EmailOptions): Promise<EmailResult>
}
```

ファイルパス: `lib/email/index.ts`

コンソールプロバイダー（開発用）

```
// ファイル: lib/email/index.ts

class ConsoleEmailProvider implements EmailProvider {
  async send(options: EmailOptions): Promise<EmailResult> {
    logger.log('≡≡≡ Email (Console) ≡≡≡')
    logger.log(`To: ${options.to}`)
    logger.log(`Subject: ${options.subject}`)
    logger.log(`HTML: ${options.html.substring(0, 200)} ... `)
    if (options.text) {
      logger.log(`Text: ${options.text}`)
    }
    logger.log('=====')
    return { success: true, messageId: `console-${Date.now()}` }
  }
}
```

ファイルパス: lib/email/index.ts

Resendプロバイダー（本番用）

```
// ファイル: lib/email/index.ts

class ResendEmailProvider implements EmailProvider {
  private client: Resend

  constructor(apiKey: string) {
    this.client = new Resend(apiKey)
  }

  async send(options: EmailOptions): Promise<EmailResult> {
    const { data, error } = await this.client.emails.send({
      from: 'BON-LOG <noreply@bon-log.com>',
      to: options.to,
      subject: options.subject,
      html: options.html,
      text: options.text,
    })

    if (error) {
      return { success: false, error: error.message }
    }

    return { success: true, messageId: data?.id }
  }
}
```

ファイルパス: `lib/email/index.ts`

期待される出力

開発環境でのメール送信:

≡ Email (Console) ≡

To: user@example.com

Subject: パスワードリセットのお願い

HTML: <!DOCTYPE html><html><head> ...

本番環境でのメール送信:

→ Resend API 経由で実際にメールが送信される

→ { success: true, messageId: "re_xxx ..." }

これで何ができるようになるか

- パスワードリセット、サブスクリプション期限通知などのメールを送信できる
- 開発環境では実際にメールを送らずコンソールで確認できる
- プロバイダーの切り替えがコード変更なしで可能

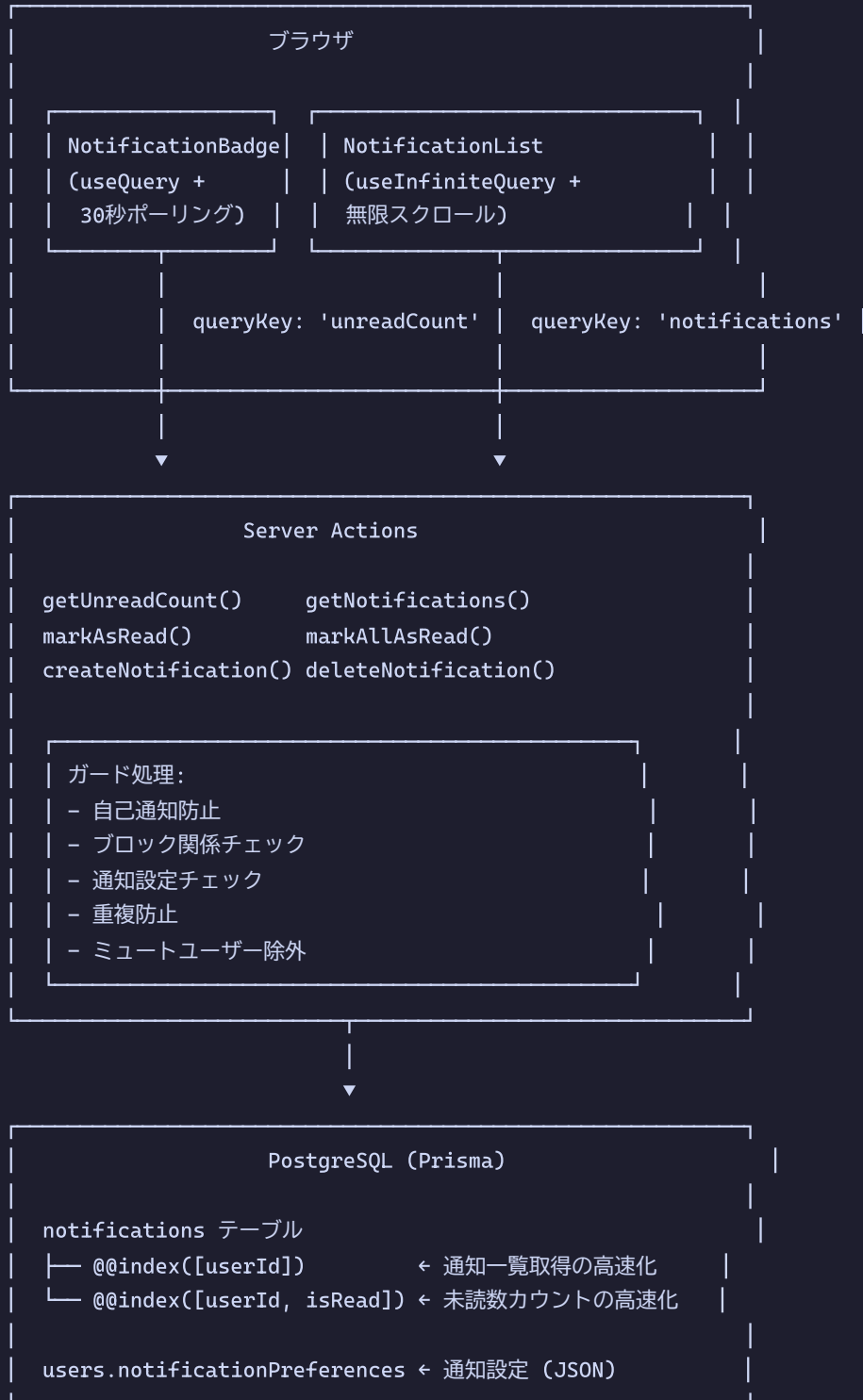
これを省略するとどうなるか

- パスワードリセットなどのメール通知が送信できない
- 開発中に本番のメールサービスを使ってしまう

13.15 全体アーキテクチャまとめ

通知システム全体の構成を図で示します。

通知システムの全体像:



主要ファイルの一覧

ファイル	役割
<code>prisma/schema.prisma</code>	NotificationType enum + Notification モデル
<code>lib/actions/notification.ts</code>	通知の CRUD Server Actions
<code>lib/actions/notification-preferences.ts</code>	通知設定の取得・更新
<code>lib/constants/limits.ts</code>	定数管理（ページサイズ、ポーリング間隔等）
<code>lib/constants/errors.ts</code>	エラーメッセージ定数
<code>lib/redis.ts</code>	Redis / インメモリキャッシュ
<code>lib/email/index.ts</code>	メール送信抽象化レイヤー
<code>components/notification/NotificationItem.tsx</code>	個別通知の表示
<code>components/notification/NotificationList.tsx</code>	通知一覧（無限スクロール）
<code>components/notification/NotificationBadge.tsx</code>	未読数バッジ
<code>app/(main)/notifications/page.tsx</code>	通知ページ（Server Component）
<code>app/(main)/notifications/loading.tsx</code>	ローディングスケルトン
<code>app/(main)/notifications/error.tsx</code>	エラーバウンダリ

13.16 技術選定の理由

なぜポーリングなのか？（WebSocketではなく）

観点	ポーリング（採用）	WebSocket
実装コスト	低い（React Queryの <code>refetchInterval</code> だけ）	高い（サーバー・クライアント両方の実装が必要）
サーバーレス対応	Vercel, Cloudflare等で問題なし	サーバーレスでは常時接続が困難
インフラコスト	低い（HTTPリクエストのみ）	高い（WebSocket接続の維持が必要）
リアルタイム性	最大30秒の遅延	ほぼリアルタイム
信頼性	高い（HTTPの標準的な仕組み）	接続断・再接続の処理が必要

BON-LOGは盆栽愛好家のSNSで、チャットアプリほどのリアルタイム性は不要です。30秒の遅延は十分許容範囲であり、ポーリングが最適な選択です。

なぜカーソルベースページネーションなのか？

観点	カーソルベース（採用）	オフセットベース
パフォーマンス	<code>O(1)</code> - インデックスで高速	<code>O(n)</code> - OFFSET分スキャン
一貫性	新規データ追加の影響なし	新規追加でずれが発生
実装	やや複雑	シンプル

通知は頻繁に追加されるため、オフセットベースだと「同じ通知が2回表示される」「通知が飛ばされる」といった問題が起きます。カーソルベースなら安定して動作します。

なぜ `@upstash/redis` なのか？

- Vercel上で動作するサーバーレス環境に最適化
- HTTP REST API を内部使用（WebSocket接続不要）

- 従量課金制でコスト効率が良い
- 公式SDKが型安全なAPIを提供

13.17 演習問題

演習1: 通知タイプの追加

`mention` タイプの通知をServer Actionsレベルで実装してみましょう。

ヒント:

1. `lib/actions/notification.ts` の `NotificationType` に `'mention'` を追加
2. `lib/actions/notification-preferences.ts` の `NotificationPreferences` に `mention` を追加
3. `components/notification/NotificationItem.tsx` の `getNotificationIcon` と `getNotificationMessage` にケースを追加

演習2: 通知の既読率を計算

全通知に対する既読率を返すServer Actionを実装してみましょう。

ヒント:

```
export async function getReadRate() {
  const session = await auth()
  if (!session?.user?.id) return { rate: 0 }

  const total = await prisma.notification.count({
    where: { userId: session.user.id },
  })
  const read = await prisma.notification.count({
    where: { userId: session.user.id, isRead: true },
  })

  return { rate: total > 0 ? Math.round((read / total) * 100) : 100 }
}
```

演習3: 通知のグルーピング

同じ投稿への複数のいいね通知を「盆栽太郎さん、他2名があなたの投稿にいいねしました」のようにグルーピングして表示する機能を考えてみましょう。

考えるポイント:

- どのレベルでグルーピングするか（DB? Server Action? コンポーネント?）
- グルーピングされた通知の既読管理はどうするか
- パフォーマンスへの影響は？

まとめ

この章では、BON-LOGの通知システムを構成する全ての要素を学びました。

学んだこと	要点
通知タイプ	Prisma enumで13種類、Server Actionsで8種類を型安全に管理
通知モデル	userId + actorId + type + postId? + commentId? のシンプルな設計
通知作成	4段階のガード（自己通知・ブロック・設定・重複）で不要な通知を防止
通知設定	ユーザーごとにJSON形式で通知タイプ別ON/OFF
通知取得	カーソルベースページネーション + ミュートフィルタリング
既読処理	個別（markAsRead）と一括（markAllAsRead）の2パターン
NotificationItem	タイプ別アイコン・メッセージ・リンク先・既読処理
NotificationList	useInfiniteQuery + SSR初期データ + 自動既読
NotificationBadge	useQuery + 30秒ポーリング + 99+表示
通知ページ	SSR + CSR + loading.tsx + error.tsx
Redis	インターフェース抽象化 + 環境別自動切り替え
メール通知	プロバイダーパターン + 環境別自動切り替え

次の章（[第14章: セキュリティ](#)）では、認証・認可やセキュリティ対策を学びます。

Generated by AI Codebase Knowledge Builder